



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA HITO 4 — Comunicación Avanzada y "Parada Temprana"

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

1. Objetivo del Hito

En este cuarto hito resolveremos el mayor desafío de la minería paralela: cuando un nodo encuentra la solución, debe avisar a los demás para que dejen de gastar CPU inútilmente.

Al finalizar esta fase, seréis capaces de:

- Detectar cuándo otro proceso ha encontrado el hash válido.
- Implementar comunicación no bloqueante en MPI.
- Finalizar correctamente todos los procesos sin desperdiciar recursos del sistema.

2. Contexto y problema a resolver

En el Hito 3 conseguisteis paralelizar el minero y hacer uso de todos los núcleos disponibles. Sin embargo, os enfrentaréis a un problema crítico de coordinación:

- **El problema:** Si un proceso (ej. *rank 1*) encuentra el *nonce* válido, los demás procesos siguen trabajando innecesariamente hasta agotar su bucle de búsqueda.
- **Pregunta guía:** ¿Cómo pueden los procesos "enterarse" de que otro ya ha terminado sin detener su propio minado?

3. Estrategia de Solución

La solución se basa en dos conceptos clave que debéis implementar:

- **Aviso global:** Un proceso debe ser capaz de comunicar al resto de la red que se ha encontrado la solución.
- **Comprobación periódica:** Los trabajadores (*workers*) no pueden detener su cálculo continuamente para ver si hay mensajes, pero sí deben comprobar cada cierto tiempo si deben parar. Los procesos trabajadores deben comprobar periódicamente (p. ej. cada 10.000 hashes) si han recibido un mensaje de "parada" del Master.
 - **"trade-off":** Si se comprueba muy a menudo, el rendimiento de minado (H/s) bajará. Si se comprueba poco, los procesos tardarán mucho en parar, gastando CPU de más.

4. Parte I — Detección de mensajes sin bloqueo

Objetivo: Permitir que un proceso escuche mensajes sin detener su ejecución.



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA HITO 4 — Comunicación Avanzada y "Parada Temprana"

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

Para ello, usaréis la función `comm.lprobe()`. Debéis modificar vuestro bucle principal para que:

1. Cada N iteraciones (por ejemplo, cada 10.000 intentos) compruebe si hay mensajes pendientes en el comunicador.
2. Si hay un mensaje de parada, el proceso rompa el bucle y termine.

Nota:

Python

```
if comm.Iprobe(source=MPI.ANY_SOURCE):  
    mensaje = comm.recv()  
    # lógica para salir del bucle
```

Antes de continuar, aseguraos de que el programa sigue minando con normalidad y no se queda congelado esperando un mensaje.

5. Parte II — Notificación de Parada

Cuando un proceso encuentra la solución, debe avisar al resto.

El proceso que encuentre la solución enviará un mensaje punto a punto no bloqueante mediante `comm.isend()` al resto de ranks. Los demás procesos comprobarán periódicamente si existe un mensaje de parada usando `comm.lprobe()` y lo recibirán con `comm.recv()`.

Nota:

Python

```
# Cuando un rank encuentra el nonce:  
for dest in range(size):  
    if dest != rank:  
        comm.isend(resultado, dest=dest, tag=STOP_TAG)
```

Y los demás procesos deben comprobar periódicamente:

```
if comm.Iprobe(source=MPI.ANY_SOURCE, tag=STOP_TAG):  
    resultado = comm.recv(source=MPI.ANY_SOURCE, tag=STOP_TAG)  
    parar = True
```

6. Validación del Sistema y Errores Comunes

Combinad ambas partes y ejecutad vuestro minero. Comprobad que vuestra solución funciona correctamente ejecutando, por ejemplo:



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA HITO 4 — Comunicación Avanzada y "Parada Temprana"

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

mpirun -n 4 python3 minero_mpi_optimizado.py --dificultad 4

Debéis observar que:

- Solo un proceso encuentra la solución.
- El resto termina rápidamente.
- La CPU en htop deja de estar al 100% tras encontrar el hash.

Errores Comunes que debéis evitar:

- **Usar recv() sin lprobe():** Esto bloqueará el proceso permanentemente esperando un mensaje que quizás nunca llegue.
- **Comprobar con demasiada frecuencia:** Hundirá vuestro rendimiento (H/s).
- **Dejar procesos huérfanos:** Aseguraos de que no quedan procesos de Python ejecutándose en segundo plano tras encontrar la solución.

7. Entregables

El grupo debe entregar en un fichero comprimido:

1. El script final `minero_mpi_optimizado.py`, que debe ser un minero local optimizado que termina limpiamente todos sus procesos en cuanto uno de ellos encuentra el hash válido.
2. Captura de pantalla donde se vea la finalización correcta de todos los procesos lanzados.
3. Breve documento (máx. 1 página) explicando:
 - Vuestra estrategia de parada.
 - La frecuencia de comprobación elegida y por qué.

8. Reflexión Final (Preparación para el Hito 5)

Este hito marca el final de la Fase I. A continuación, expandiremos el clúster a múltiples máquinas físicas/virtuales.



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA HITO 4 — Comunicación Avanzada y "Parada Temprana"

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

- **Piensa:** ¿Qué pasará cuando los procesos estén en máquinas distintas conectadas por red? ¿Cómo afectará la latencia de la red a esa frecuencia de comprobación de 10.000 iteraciones que habéis elegido hoy?